

1. Setup e Imports

```
import json as _json
import json
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm
import os
import sys
from pathlib import Path
import subprocess

# Agregar path del proyecto para imports
sys.path.append(os.path.abspath('../..'))
from sae.tools.naming_utils import save_checkpoint, get_model_name, get_extras_id, get_checkpoint_dir, get_model_dir
from sae.tools.experiment_utils import get_experiment_dir, get_plots_dir, get_metrics_dir, get_logs_dir

# Configuración de visualización para Quarto/PDF
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', None)
np.set_printoptions(linewidth=100, precision=4, suppress=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Dispositivo: {device}')
```

Dispositivo: cuda

2 Metadata para Naming de Checkpoints

```
# Metadata para naming de checkpoints (separada de hiperparámetros)
NAMING_META = {
    'variante': 'matryoshkabatchtopk',
    'tecnica': 'matryoshkabatchtopk',
    'capa': 6,
    'juegos': 5000,
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt',
    'experiment_base_path': os.path.abspath('../../../experiments'),
    'extras': {
        'expansion': 16,
        'k': 81,
        'matryoshka_widths': [2048, 4096, 8192],
        'epochs': 1000,
        'patience': 20,
        'lr': 1e-3
    }
}

print('\nMetadata de naming:')
for key, value in NAMING_META.items():
    print(f' {key}: {value}')
```

Metadata de naming:

```
variante: matryoshkabatchtopk
tecnica: matryoshkabatchtopk
capa: 6
juegos: 5000
base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments
extras: {'expansion': 16, 'k': 81, 'matryoshka_widths': [2048, 4096, 8192], 'epochs': 1000
```

```
extras_id = get_extras_id(
    get_checkpoint_dir(
        NAMING_META['base_path'],
        NAMING_META['variante'],
        NAMING_META['tecnica'],
        NAMING_META['capa'],
        NAMING_META['juegos']
```

```

    ),
    NAMING_META['extras']
) if NAMING_META.get('extras') else None

# get_plots_dir crea el directorio automáticamente
plots_dir = Path(get_plots_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
))

print(f'extras_id: {extras_id}')
print(f'plots_dir: {plots_dir}')

```

Nuevo extras registrado: ex2 → {'expansion': 16, 'k': 81, 'matryoshka_widths': [2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144, 524288, 1048576, 2097152, 4194304, 8388608, 16777216, 33554432, 67108864, 134217728, 268435456, 536870912, 1073741824, 2147483648, 4294967296, 8589934592, 17179869184, 34359738368, 68719476736, 137438953472, 274877906944, 549755813888, 1099511627776, 2199023255552, 4398046511104, 8796093022208, 17592186044416, 35184372088832, 70368744177664, 140737488355328, 281474976710656, 562949953421312, 1125899906842624, 2251799813685248, 4503599627370496, 9007199254740992, 18014398509481984, 36028797018963968, 72057594037927936, 144115188075855872, 288230376151711744, 576460752303423488, 1152921504606846976, 2305843009213693952, 4611686018427387904, 9223372036854775808, 18446744073709551616, 36893488147419103232, 73786976294838206464, 147573952589676412928, 295147905179352825856, 590295810358705651712, 1180591620717411303424, 2361183241434822606848, 4722366482869645213696, 9444732965739290427392, 18889465931478580854784, 37778931862957161709568, 75557863725914323419136, 151115727451828646838272, 302231454903657293676544, 604462909807314587353088, 1208925819614629174706176, 2417851639229258349412352, 4835703278458516698824704, 9671406556917033397649408, 19342813113834066795298816, 38685626227668133590597632, 77371252455336267181195264, 154742504910672534362390528, 309485009821345068724781056, 618970019642690137449562112, 1237940039285380274899124224, 2475880078570760549798248448, 4951760157141521099596496896, 9903520314283042199192993792, 19807040628566084398385987584, 39614081257132168796771975168, 79228162514264337593543950336, 158456325028528675187087900672, 316912650057057350374175801344, 633825300114114700748351602688, 1267650600228229401496703205376, 2535301200456458802993406410752, 5070602400912917605986812821504, 10141204801825835211973625643008, 20282409603651670423947251286016, 40564819207303340847894502572032, 81129638414606681695789005144064, 162259276829213363391578010288128, 324518553658426726783156020576256, 649037107316853453566312041152512, 1298074214633706907132624082305024, 2596148429267413814265248164610048, 5192296858534827628530496329220096, 10384593717069655257060992658440192, 20769187434139310514121985316880384, 41538374868278621028243970633760768, 83076749736557242056487941267521536, 166153499473114484112975882535043072, 332306998946228968225951765070086144, 664613997892457936451903530140172288, 1329227995784915872903807060280344576, 2658455991569831745807614120560689152, 5316911983139663491615228241121378304, 10633823966279326983230456482242756608, 21267647932558653966460912964485513216, 42535295865117307932921825928971026432, 85070591730234615865843651857942052864, 170141183460469231731687303715884105728, 340282366920938463463374607431768211456, 680564733841876926926749214863536422912, 1361129467683753853853498429727072845824, 2722258935367507707706996859454145691648, 5444517870735015415413993718908291383296, 10889035741470030830827987437816582766592, 21778071482940061661655974875633165533184, 43556142965880123323311949751266331066368, 87112285931760246646623899502532662132736, 174224571863520493293247799005065324265472, 348449143727040986586495598010130648530944, 696898287454081973172991196020261297061888, 1393796574908163946345982392040522594123776, 2787593149816327892691964784081045188247552, 5575186299632655785383929568162090376495104, 11150372599265311570767859136324180752990208, 22300745198530623141535718272648361505980416, 44601490397061246283071436545296723011960832, 89202980794122492566142873090593446023921664, 178405961588244985132285746181186892047843328, 356811923176489970264571492362373784095686656, 713623846352979940529142984724747568191373312, 1427247692705959881058285969449495136382746624, 2854495385411919762116571938898990272765493248, 5708990770823839524233143877797980545530986496, 11417981541647679048466287755595961091061972992, 22835963083295358096932575511191922182123945984, 45671926166590716193865151022383844364247891968, 91343852333181432387730302044767688728495783936, 182687704666362864775460604089535377456991567872, 365375409332725729550921208179070754913983135744, 730750818665451459101842416358141509827966271488, 1461501637330902918203684832716283019655932542976, 2923003274661805836407369665432566039311865085952, 5846006549323611672814739330865132078623730171904, 11692013098647223345629478661730264157247460343808, 23384026197294446691258957323460528314494920687616, 46768052394588893382517914646921056628989841375232, 93536104789177786765035829293842113257979682750464, 187072209578355573530071658587684226515959365500928, 374144419156711147060143317175368453031918731001856, 748288838313422294120286634350736906063837462003712, 1496577676626844588240573268701473812127674924007424, 2993155353253689176481146537402947624255349848014848, 5986310706507378352962293074805895248510699696029696, 11972621413014756705924586149611790497021399392059392, 23945242826029513411849172299223580994042798784118784, 47890485652059026823698344598447161988085597568237568, 95780971304118053647396689196894323976171195136475136, 191561942608236107294793378393788647952342390272950272, 383123885216472214589586756787577295904684780545900544, 766247770432944429179173513575154591809369561091801088, 1532495540865888858358347027150309183618739122183602176, 3064991081731777716716694054300618367237478244367204352, 6129982163463555433433388108601236734474956488734408704, 12259964326927110866866776217202473468949912977468817408, 24519928653854221733733552434404946937899825954937634816, 49039857307708443467467104868809893875799651909875269632, 98079714615416886934934209737619787751599303819750539264, 196159429230833773869868419475239575503198607639501078528, 392318858461667547739736838950479151006397215279002157056, 784637716923335095479473677900958302012794430558004314112, 1569275433846670190958947355801916604025588861116008628224, 3138550867693340381917894711603833208051177722232017256448, 6277101735386680763835789423207666416102355444464034512896, 12554203470773361527671578846415332832204710888928069025792, 25108406941546723055343157692830665664409421777856138051584, 50216813883093446110686315385661331328818843555712276103168, 100433627766186892221372630771322662657637687111424552206336, 200867255532373784442745261542645325315275374222849104412672, 401734511064747568885490523085290650630550748445698208825344, 803469022129495137770981046170581301261101496891396417650688, 1606938044258990275541962092341162602522202993782792835301376, 3213876088517980551083924184682325205044405987565585670602752, 6427752177035961102167848369364650410088811975131171341205504, 12855504354071922204335696738729300820177623950262342682411008, 25711008708143844408671393477458601640355247900524685364822016, 51422017416287688817342786954917203280710495801049370729644032, 102844034832575377634685573909834406561420991602098741459288064, 205688069665150755269371147819668813122841983204197482918576128, 411376139330301510538742295639337626245683966408394965837152256, 822752278660603021077484591278675252491367932816789931674304512, 1645504557321206042154969182557350504982735865633579863348609024, 3291009114642412084309938365114701009965471731267159726697218048, 6582018229284824168619876730229402019930943462534319453394436096, 13164036458569648337239753460458804039861886925068638906788872192, 26328072917139296674479506920917608079723773850137277813577744384, 52656145834278593348959013841835216159447547700274555627155488768, 105312291668557186697918027683670432318895095400549111254310977536, 210624583337114373395836055367340864637790190801098222508621955072, 421249166674228746791672110734681729275580381602196445017243910144, 842498333348457493583344221469363458551160763204392890034487820288, 1684996666696914987166688442938726917102321526408785780068975640576, 3369993333393829974333376885877453834204643052817571560137951281152, 6739986666787659948666753771754907668409286105635143120275902562304, 13479973333575319897333507543509815336818572211270286240551805124608, 26959946667150639794667015087019630673637144422540572481103610249216, 53919893334301279589334030174039261347274288845081144962207220498432, 107839786668602559178668060348078522694548577690162289924414440996864, 215679573337205118357336120696157045389097155380324579848828881993728, 431359146674410236714672241392314090778194310760649159697657763987456, 862718293348820473429344482784628181556388621521298319395315527974912, 1725436586697640946858688965569256363112777243042596638790631055949824, 3450873173395281893717377931138512726225554486085193277581262111899648, 6901746346790563787434755862277025452451108972170386555162524223799296, 13803492693581127574869511724554050904902217944340773110325048447598592, 27606985387162255149739023449108101809804435888681546220650096895197184, 55213970774324510299478046898216203619608871777363092441300193790394368, 110427941548649020598956093796432407239217743554726184882600387580788736, 220855883097298041197912187592864814478435487109452369765200775161577472, 441711766194596082395824375185729628956870974218904739530401550323154944, 883423532389192164791648750371459257913741948437809479060803100646309888, 1766847064778384329583297500742918515827483896875618958121606201292619776, 3533694129556768659166595001485837031654967793751237916243212402585239552, 7067388259113537318333190002971674063309935587502475832486424805170479104, 14134776518227074636666380005943348126619871175004951664972849610340958208, 28269553036454149273332760011886696253239742350009903329945699220681916416, 56539106072908298546665520023773392506479484700019806659891398441363832832, 113078212145816597093331040047546785012958969400039613319782796882727665664, 226156424291633194186662080095093570025917938800079226639565593765455331328, 452312848583266388373324160190187140051835877600158453279131187530910662656, 904625697166532776746648320380374280103671755200316906558262375061821325312, 1809251394333065553493296640760748560207343510400633813116524750123642650624, 3618502788666131106986593281521497120414687020801267626233049500247285301248, 7237005577332262213973186563042994240829374041602535252466099000494570602496, 14474011154664524427946373126085988481658748083205070504932198000989141204992, 28948022309329048855892746252171976963317496166410141009864396001978282409984, 57896044618658097711785492504343953926634992332820282019728792003956564819968, 115792089237316195423570985008687907853269984665640564039457584007913129639936, 231584178474632390847141970017375815706539969331281128078915168015826259279872, 463168356949264781694283940034751631413079938662562256157830336031652518559744, 926336713898529563388567880069503262826159877325124512315660672063305037119488, 1852673427797059126777135760139006525652319754650249024631321344126610074238976, 3705346855594118253554271520278013051304639509300498049262642688253220148477952, 7410693711188236507108543040556026102609279018600996098525285376506440296955904, 14821387422376473014217086081112052205218558037201992197050570753012880593911808, 29642774844752946028434172162224104410437116074403984394101141506025761187823616, 59285549689505892056868344324448208820874232148807968788202283012051522375647232, 118571099379011784113736688648896417641748464297615937576404566024103044751294464, 237142198758023568227473377297792835283496928595231875152809132048206089502588928, 474284397516047136454946754595585670566993857190463750305618264096412179005177856, 948568795032094272909893509191171341133987714380927500611236528192824358010355712, 1897137590064188545819787018382342682267975428761855001222473056385648716020711424, 3794275180128377091639574036764685364535950857523710002444946112771297432041422848, 7588550360256754183279148073529370729071901715047420004889892225542594864082845696, 15177100720513508366558296147058741458143803430094840009779784451085189728165691392, 30354201441027016733116592294117482916287606860189680019559568902170379456331382784, 60708402882054033466233184588234965832575213720379360039119137804340758912662765568, 121416805764108066932466369176469931665150427440758720078238275608681517825325531136, 24283361152

```

NUM_EPOCHS = NAMING_META['extras']['epochs']

cfg = {
    'architecture': 'matryoshkabatchtopk',
    'd_in': 512,
    'd_sae': 8192,
    'k': 81,
    'matryoshka_widths': [2048, 4096, 8192],
    'lr': 1e-3,
    'train_batch_size_tokens': 4096,
    'hook_name': 'blocks.5.hook_resid_post',
    # P3: betas según config.py de los autores (beta2=0.99 vs default PyTorch 0.999)
    'beta1': 0.9,
    'beta2': 0.99,
    # P1: parámetros para auxiliary loss de dead features (config.py de los autores)
    'n_batches_to_dead': 20, # batches sin activación para declarar feature muerta
    'top_k_aux': 512, # top-k features muertas usadas en L_aux
    'aux_penalty': 1/32, # peso de L_aux (paper ec. 5)
    # P2: normalización del input por muestra
    'input_unit_norm': True,
}

cfg_summary = pd.DataFrame({
    'Parámetro': ['architecture', 'd_in', 'd_sae', 'k (L0 promedio)', 'matryoshka_widths',
                  'hook_name', 'lr', 'beta1', 'beta2',
                  'n_batches_to_dead', 'top_k_aux', 'aux_penalty', 'input_unit_norm'],
    'Valor': [cfg['architecture'], cfg['d_in'], cfg['d_sae'], cfg['k'],
              str(cfg['matryoshka_widths']), cfg['hook_name'], cfg['lr'],
              cfg['beta1'], cfg['beta2'],
              cfg['n_batches_to_dead'], cfg['top_k_aux'], cfg['aux_penalty'],
              cfg['input_unit_norm']]
})

print('Configuración SAELens:')
print(cfg_summary.to_string(index=False))

```

Configuración SAELens:

Parámetro	Valor
architecture	matryoshkabatchtopk
d_in	512
d_sae	8192
k (L0 promedio)	81
matryoshka_widths	[2048, 4096, 8192]

hook_name	blocks.5.hook_resid_post
lr	0.001
beta1	0.9
beta2	0.99
n_batches_to_dead	20
top_k_aux	512
aux_penalty	0.03125
input_unit_norm	True

4. Dataset de Activaciones (Train/Validation Split 80-20%)

Activaciones pre-extraídas con `sae/activations/run_extraction.py` desde la capa `blocks.5.hook_resid_post`.

```
class ActivationsDataset(Dataset):
    """Dataset para cargar activaciones extraídas"""
    def __init__(self, activations_path):
        self.activations = np.load(activations_path, mmap_mode='r')
    def __len__(self):
        return len(self.activations)
    def __getitem__(self, idx):
        return torch.tensor(self.activations[idx], dtype=torch.float32)

activations_file = Path(f"../../activations/data/layer{NAMING_META['capa']}_{NAMING_META['hook']}")
full_dataset = ActivationsDataset(activations_file)

# Split 80-20 reproducible
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(
    full_dataset, [train_size, val_size],
    generator=torch.Generator().manual_seed(42)
)

train_loader = DataLoader(train_dataset, batch_size=cfg["train_batch_size_tokens"], shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=cfg["train_batch_size_tokens"], shuffle=False)

dataset_info = pd.DataFrame({
    'Conjunto': ['Train', 'Validación', 'Total'],
    'Muestras': [f'{len(train_dataset):,}', f'{len(val_dataset):,}', f'{len(full_dataset):,}'],
    'Shape': [str(full_dataset.activations.shape), '', ''],
})
```

```
})
print(dataset_info.to_string(index=False))
```

	Conjunto	Muestras	Shape
	Train	236,000	(295000, 512)
Validación		59,000	
	Total	295,000	

5. Modelo: MatryoshkaBatchTopK SAE

Implementación de la arquitectura MatryoshkaBatchTopK según el paper (ec. 3-5) y el código de los autores (`sae.py`).

Arquitectura: - El encoder proyecta **una sola vez** con W_{enc} completo ($d_{in} \rightarrow d_{sae}$) para obtener $f(x)$ de tamaño d_{sae} (paper ec. 3) - **BatchTopK global único:** se seleccionan los $k \times batch_size$ valores más altos sobre los $d_{sae} \times batch_size$ valores del batch completo — un único threshold compartido entre todos los anchos (paper ec. 6) - Las reconstrucciones por nivel usan **prefijos del mismo vector:** `all_acts_topk[:, 0:w]` — las features 0:2048 son literalmente las mismas en el nivel $w=2048$ y en el nivel $w=4096$ (paper ec. 4) - En **eval**, se usa un threshold persistente calibrado con EMA durante training — no se recalcula (paper ec. 7) - **L0 es variable por muestra pero promedia k en cada batch**

Parámetro	L1 (standard)	BatchTopK	Matryoshka-BatchTopK
Sparsity	<code>sparsity_coef</code> (soft)	k (promedio del batch)	k (promedio, threshold global)
L0	Variable	Variable por muestra	Variable por muestra
Loss	$MSE + \cdot L1$	MSE	$\Sigma MSE_width + \cdot L_aux$

```
class MatryoshkaBatchTopKSAE(nn.Module):
    """
    Sparse Autoencoder con activación MatryoshkaBatchTopK.
    Corregido según paper (ec. 3-5) e implementación de los autores (sae.py).

    Correcciones aplicadas:
    - P1: encoding único con  $W_{enc}$  completo; BatchTopK global; prefijos compartidos
    - P1: BatchTopK preserva valores originales ( $x * indicator$ ), threshold EMA para eval
```

```

- P1: auxiliary loss para dead features (sae.py:198-221)
- P2: make_decoder_weights_and_grad_unit_norm proyecta gradiente (sae.py:51-57)
- P2: preprocessing input_unit_norm configurable (sae.py:36-43)
- P3: W_dec inicializado como transpuesta de W_enc (sae.py:93-94)
"""
def __init__(self, d_in, d_sae, k, matryoshka_widths,
              n_batches_to_dead=20, top_k_aux=512, aux_penalty=1/32,
              input_unit_norm=False):
    super().__init__()
    self.d_in = d_in
    self.d_sae = d_sae
    self.k = k
    self.matryoshka_widths = sorted(matryoshka_widths)
    assert self.matryoshka_widths[-1] == d_sae, \
        f"El último ancho ({self.matryoshka_widths[-1]}) debe ser d_sae ({d_sae})"

    self.n_batches_to_dead = n_batches_to_dead
    self.top_k_aux = top_k_aux
    self.aux_penalty = aux_penalty
    self.input_unit_norm = input_unit_norm

    self.W_enc = nn.Parameter(torch.empty(d_in, d_sae))
    self.W_dec = nn.Parameter(torch.empty(d_sae, d_in))
    self.b_dec = nn.Parameter(torch.zeros(d_in))

    # P3: W_dec = W_enc^T normalizado (sae.py:93-94) - inicialización tied
    nn.init.kaiming_uniform_(self.W_enc)
    self.W_dec.data[:] = self.W_enc.t().data
    self.W_dec.data[:] = self.W_dec / self.W_dec.norm(dim=-1, keepdim=True)

    # P1: threshold persistente calibrado con EMA durante training (sae.py:97)
    self.register_buffer('threshold', torch.tensor(0.0))
    # P1: contador de batches sin activación por feature - para L_aux (sae.py:30-32)
    self.register_buffer('num_batches_not_active', torch.zeros(d_sae))

# -----
# P2: Preprocessing del input (sae.py:36-48)
# -----

def preprocess_input(self, x):
    if self.input_unit_norm:
        x_mean = x.mean(dim=-1, keepdim=True)

```

```

        x = x - x_mean
        x_std = x.std(dim=-1, keepdim=True)
        x = x / (x_std + 1e-5)
        return x, x_mean, x_std
    return x, None, None

def postprocess_output(self, x_reconstruct, x_mean, x_std):
    if self.input_unit_norm and x_mean is not None:
        x_reconstruct = x_reconstruct * x_std + x_mean
    return x_reconstruct

# -----
# P1: Encoding único + BatchTopK global (sae.py:100-119)
# -----

def compute_activations(self, x_cent):
    # P1: encoding ÚNICO con W_enc completo - un solo pase para todos los anchos
    # Las features 0:w son literalmente las mismas en todos los niveles (paper ec. 3)
    pre_acts = x_cent @ self.W_enc      # [batch, d_sae]
    acts      = F.relu(pre_acts)        # ReLU antes del topk (sae.py:102)

    if self.training:
        # P1: BatchTopK global sobre d_sae*batch_size valores (paper ec. 6)
        # Preserva valores originales con scatter - NO resta threshold (sae.py:104-114)
        topk_result = torch.topk(acts.flatten(), self.k * x_cent.shape[0])
        acts_topk = (
            torch.zeros_like(acts.flatten())
            .scatter(-1, topk_result.indices, topk_result.values)
            .reshape(acts.shape)
        )
        self._update_threshold(acts_topk)
    else:
        # P1: eval usa threshold persistente, no recalcula (paper ec. 7, sae.py:117)
        acts_topk = torch.where(acts > self.threshold, acts, torch.zeros_like(acts))

    return acts, acts_topk

@torch.no_grad()
def _update_threshold(self, acts_topk, lr=0.01):
    # P1: EMA del mínimo valor positivo - calibra para eval (sae.py:224-228)
    positive_mask = acts_topk > 0
    if positive_mask.any():

```



```

        min_positive = acts_topk[positive_mask].min()
        self.threshold = (1 - lr) * self.threshold + lr * min_positive

@torch.no_grad()
def _update_inactive_features(self, acts_topk):
    # P1: necesario para L_aux - trackea qué features no se activan (sae.py:60-62)
    self.num_batches_not_active += (acts_topk.sum(0) == 0).float()
    self.num_batches_not_active[acts_topk.sum(0) > 0] = 0

# -----
# Forward
# -----

def forward(self, x):
    """
    Retorna:
        outputs      : lista de (recon_w, hidden_w) para cada ancho matryoshka
                       recon_w está en espacio normalizado (= espacio original si input_unit_norm=True)
        all_acts      : pre-topk [batch, d_sae] - para L_aux
        all_acts_topk: post-topk [batch, d_sae]
        x_proc        : x preprocesado (= x si input_unit_norm=False)
    """
    x_proc, x_mean, x_std = self.preprocess_input(x)
    x_cent = x_proc - self.b_dec
    all_acts, all_acts_topk = self.compute_activations(x_cent)

    # P1: reconstrucciones por PREFIJO compartido (paper ec. 4, sae.py:149-155)
    # all_acts_topk[:, 0:w] son las mismas features en todos los niveles anidados
    # recon_w se mantiene en espacio normalizado para que la loss compare espacios iguales
    outputs = []
    for w in self.matryoshka_widths:
        hidden_w = all_acts_topk[:, :w]
        recon_w = hidden_w @ self.W_dec[:, :w] + self.b_dec
        outputs.append((recon_w, hidden_w))

    self._update_inactive_features(all_acts_topk)
    return outputs, all_acts, all_acts_topk, x_proc

# -----
# P1: Auxiliary loss para dead features (sae.py:198-221)
# -----

```

```

def get_auxiliary_loss(self, x_proc, x_reconstruct_full, all_acts):
    residual = x_proc.float() - x_reconstruct_full.float()
    aux_reconstruct = torch.zeros_like(residual)

    dead_features = self.num_batches_not_active >= self.n_batches_to_dead

    if dead_features.sum() > 0:
        acts_topk_aux = torch.topk(
            all_acts[:, dead_features],
            min(self.top_k_aux, int(dead_features.sum()))),
            dim=-1,
        )
        acts_aux = torch.zeros_like(all_acts[:, dead_features]).scatter(
            -1, acts_topk_aux.indices, acts_topk_aux.values
        )
        aux_reconstruct = aux_reconstruct + acts_aux @ self.W_dec[dead_features]

    if aux_reconstruct.abs().sum() > 0:
        return self.aux_penalty * (aux_reconstruct.float() - residual.float()).pow(2).mean()
    return torch.tensor(0.0, device=x_proc.device)

# -----
# P2: Constraint de norma unitaria con proyección del gradiente (sae.py:51-57)
# -----

@torch.no_grad()
def make_decoder_weights_and_grad_unit_norm(self):
    # P2: proyecta gradiente al espacio tangente de la esfera unitaria + normaliza
    # @torch.no_grad() evita trazar grafo interno - no impide leer .grad (sae.py:50)
    # Llamar ANTES de optimizer.step() (training.py:31)
    W_dec_normed = self.W_dec / self.W_dec.norm(dim=-1, keepdim=True)
    W_dec_grad_proj = (
        (self.W_dec.grad * W_dec_normed).sum(-1, keepdim=True) * W_dec_normed
    )
    self.W_dec.grad -= W_dec_grad_proj
    self.W_dec.data = W_dec_normed

# Crear modelo
model = MatryoshkaBatchTopKSAE(
    d_in=cfg['d_in'],
    d_sae=cfg['d_sae'],

```

```

k=cfg['k'],
matryoshka_widths=cfg['matryoshka_widths'],
n_batches_to_dead=cfg['n_batches_to_dead'],
top_k_aux=cfg['top_k_aux'],
aux_penalty=cfg['aux_penalty'],
input_unit_norm=cfg['input_unit_norm'],
).to(device)
num_params = sum(p.numel() for p in model.parameters())

model_info = pd.DataFrame({
    'Componente': [
        'Input', 'Encoder (W_enc)', 'Activación', 'Anchos matryoshka',
        'Hidden activos (por nivel)', 'Decoder (W_dec)', 'Output', 'Total params'
    ],
    'Dimensión': [
        f'{cfg["d_in"]}',
        f'{cfg["d_in"]} -> {cfg["d_sae"]} (único pase, prefijos compartidos)',
        f'BatchTopK global (k={cfg["k"]}) - threshold EMA en eval',
        str(cfg['matryoshka_widths']),
        f' {cfg["k"]} activos (promedio del batch)',
        f'{cfg["d_sae"]} -> {cfg["d_in"]} (slices de prefijo)',
        f'{cfg["d_in"]}',
        f'{num_params:,}'
    ]
})
print(model_info.to_string(index=False))

```

Componente	Dimensión
Input	512
Encoder (W_enc)	512 -> 8192 (único pase, prefijos compartidos)
Activación	BatchTopK global (k=81) - threshold EMA en eval
Anchos matryoshka	[2048, 4096, 8192]
Hidden activos (por nivel)	81 activos (promedio del batch)
Decoder (W_dec)	8192 -> 512 (slices de prefijo)
Output	512
Total params	8,389,120

6. Función de Pérdida MatryoshkaBatchTopK

Loss = mean MSE(recon_w, x) para cada ancho w matryoshka_widths +
 · L_aux

- **MSE en cada nivel:** error de reconstrucción desde cada sub-espacio matryoshka
- **Sin L1 penalty:** la sparsity está controlada por el threshold global de BatchTopK
- **Loss total = media de MSE en todos los anchos** — mantiene la escala estable y preserva el balance entre reconstrucción y aux_loss independientemente del número de anchos (alineado con Bussmann sae.py:172)
- La loss anidada fuerza a que los sub-espacios más pequeños también sean representaciones completas del input

```
def loss_function(x_proc, outputs, all_acts, model):
    """
    Pérdida MatryoshkaBatchTopK: media de MSE en cada ancho + auxiliary loss.

    Paper ec. 5:  $L(x) = \sum_{m \in M} \|x - \hat{x}\|^2 + \lambda L_{aux}$ 
    Bussmann (sae.py): promedia dividiendo por n_grupos - mantiene la escala de
    la MSE estable independientemente del número de anchos, y preserva el balance
    entre reconstrucción y aux_loss.

    Args:
        x_proc: activaciones preprocesadas [batch, d_in] (= x si input_unit_norm=False)
        outputs: lista de (recon_w, hidden_w) por ancho matryoshka
        all_acts: activaciones pre-topk [batch, d_sae] - para L_aux
        model: referencia al modelo (para get_auxiliary_loss)

    Returns:
        total_loss, mse_per_width, aux_loss
    """
    x_reconstruct_full = outputs[-1][0] # reconstrucción del ancho máximo

    # Media de MSE en todos los anchos + término b_dec (alineado con Bussmann sae.py:163-172)
    mse_bdec = F.mse_loss(model.b_dec.expand_as(x_proc), x_proc)
    mse_per_width = [F.mse_loss(recon_w, x_proc) for recon_w, _ in outputs]
    mean_mse = (mse_bdec + sum(mse_per_width)) / (len(mse_per_width) + 1)

    # P1: auxiliary loss para dead features (paper ec. 5:  $\lambda L_{aux}$ , sae.py:198-221)
    aux_loss = model.get_auxiliary_loss(x_proc, x_reconstruct_full, all_acts)

    return mean_mse + aux_loss, mse_per_width, aux_loss

# P3: betas según config.py de los autores - beta2=0.99 vs default PyTorch 0.999
# (config.py: beta1=0.9, beta2=0.99)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=cfg["lr"],
```

```

    betas=(cfg["beta1"], cfg["beta2"])
)

optim_info = pd.DataFrame({
    'Parámetro': ['Optimizador', 'lr', 'beta1', 'beta2',
                  'k (L0 promedio)', 'Anchos matryoshka',
                  'aux_penalty', 'n_batches_to_dead', 'top_k_aux'],
    'Valor':      ['Adam', cfg["lr"], cfg["beta1"], cfg["beta2"],
                  cfg["k"], str(cfg['matryoshka_widths']),
                  cfg['aux_penalty'], cfg['n_batches_to_dead'], cfg['top_k_aux']]
})
print(optim_info.to_string(index=False))

```

Parámetro	Valor
Optimizador	Adam
lr	0.001
beta1	0.9
beta2	0.99
k (L0 promedio)	81
Anchos matryoshka	[2048, 4096, 8192]
aux_penalty	0.03125
n_batches_to_dead	20
top_k_aux	512

7. Loop de Entrenamiento

```

PATIENCE = NAMING_META['extras']['patience']
WIDTHS    = cfg['matryoshka_widths']

save_dir = Path('./saved_model')
save_dir.mkdir(exist_ok=True)
best_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='best',
    extras_id=extras_id
)
BEST_CKPT = save_dir / best_model_name

```

```

history = {
    'train_total_loss': [],
    'train_aux_loss': [],
    'train_l0_sparsity': [],
    'train_fraction_active': [],
    'val_total_loss': [],
    'val_aux_loss': [],
    'val_l0_sparsity': [],
    'val_fraction_active': [],
}
for w in WIDTHS:
    history[f'val_mse_w{w}'] = []

best_val_loss = float('inf')
best_epoch = 0
epochs_no_improve = 0

print(f'Iniciando entrenamiento: {NUM_EPOCHS} épocas máximo')
print(f'Early stopping: patience={PATIENCE}')
print(f'k = {cfg["k"]} features activas (L0 promedio k por batch)')
print(f'Anchos matryoshka: {WIDTHS}')
print(f'Best checkpoint: {BEST_CKPT}')
print('=' * 70)

for epoch in range(NUM_EPOCHS):

    # ----- TRAIN
    model.train()
    t_total = t_aux = t_l0 = t_frac = 0.0

    for batch in train_loader:
        x = batch.to(device)
        optimizer.zero_grad()

        # P1: forward devuelve (outputs, all_acts, all_acts_topk, x_proc)
        outputs, all_acts, all_acts_topk, x_proc = model(x)
        total_loss, _, aux_loss_val = loss_function(x_proc, outputs, all_acts, model)
        total_loss.backward()

        # P2: proyectar gradiente al espacio tangente de la esfera + normalizar
        # ANTES de optimizer.step() (sae.py:51-57, training.py:31)
        model.make_decoder_weights_and_grad_unit_norm()

```

```

optimizer.step()
# REMOVED: model.normalize_decoder() - reemplazado por make_decoder_weights_and_grad

with torch.no_grad():
    hidden_full = outputs[-1][1]
    t_total += total_loss.item()
    t_aux    += aux_loss_val.item()
    t_l0     += (hidden_full > 0).float().sum(dim=1).mean().item()
    t_frac   += (hidden_full > 0).float().mean().item()

n = len(train_loader)
history['train_total_loss'].append(t_total / n)
history['train_aux_loss'].append(t_aux / n)
history['train_l0_sparsity'].append(t_l0 / n)
history['train_fraction_active'].append(t_frac / n)

# ----- VALIDATION
model.eval()
v_total = v_aux = v_l0 = v_frac = 0.0
v_per_width = {w: 0.0 for w in WIDTHHS}

with torch.no_grad():
    for batch in val_loader:
        x = batch.to(device)
        outputs, all_acts, all_acts_topk, x_proc = model(x)
        total_loss, mse_per_width, aux_loss_val = loss_function(x_proc, outputs, all_acts)
        hidden_full = outputs[-1][1]

        v_total += total_loss.item()
        v_aux    += aux_loss_val.item()
        v_l0     += (hidden_full > 0).float().sum(dim=1).mean().item()
        v_frac   += (hidden_full > 0).float().mean().item()
        for w, mse_w in zip(WIDTHHS, mse_per_width):
            v_per_width[w] += mse_w.item()

nv = len(val_loader)
history['val_total_loss'].append(v_total / nv)
history['val_aux_loss'].append(v_aux / nv)
history['val_l0_sparsity'].append(v_l0 / nv)
history['val_fraction_active'].append(v_frac / nv)
for w in WIDTHHS:
    history[f'val_mse_w{w}'].append(v_per_width[w] / nv)

```

```

width_str = ' | '.join(f'w{w}={history[f"val_mse_w{w}"][-1]:.4f}' for w in WIDTHS)
print(f"Época {epoch+1:3d}/{NUM_EPOCHS} | "
      f"Train: {history['train_total_loss'][-1]:.6f} | "
      f"Val: {history['val_total_loss'][-1]:.6f} | "
      f"L_aux: {history['val_aux_loss'][-1]:.6f} | "
      f"L0: {history['val_l0_sparsity'][-1]:.1f} | "
      f"{width_str}")

# Early stopping (basado en loss total de validación)
if history['val_total_loss'][-1] < best_val_loss - 1e-4:
    best_val_loss = history['val_total_loss'][-1]
    best_epoch = epoch + 1
    epochs_no_improve = 0
    torch.save(model.state_dict(), BEST_CKPT)
    save_checkpoint(
        model,
        base_path=NAMING_META['base_path'],
        variante=NAMING_META['variante'],
        tecnica=NAMING_META['tecnica'],
        capa=NAMING_META['capa'],
        juegos=NAMING_META['juegos'],
        estado='best',
        extras=NAMING_META['extras'],
        config=cfg,
        history=history,
        optimizer=optimizer
    )
    print(f' Mejor modelo guardado (época {best_epoch} | Val Loss: {best_val_loss:.6f})

else:
    epochs_no_improve += 1
    if epochs_no_improve >= PATIENCE:
        print(f'\nEarly stopping en época {epoch+1} (mejor: época {best_epoch})')
        break

# Restaurar mejor modelo
model.load_state_dict(torch.load(BEST_CKPT, weights_only=True))
print(f'\nMejor época: {best_epoch} | Mejor Val Loss: {best_val_loss:.6f}')

```

Iniciando entrenamiento: 1000 épocas máximo
 Early stopping: patience=20

k = 81 features activas (L0 promedio k por batch)
 Anchos matryoshka: [2048, 4096, 8192]
 Best checkpoint: saved_model\sae_matryoshkabatchtopk_matryoshkabatchtopk_l6_5000g_ex2_best.p
 =====

Época	Train	Val	L_aux	L0	w2048
1/1000	0.570653	1.997144	0.000000	953.0	0.975
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 1 Val Loss: 1.997144)					
2/1000	0.485447	0.605642	0.000000	284.5	0.331
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 2 Val Loss: 0.605642)					
3/1000	0.461714	0.450777	0.000000	149.8	0.275
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 3 Val Loss: 0.450777)					
4/1000	0.448173	0.437321	0.000000	109.7	0.279
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 4 Val Loss: 0.437321)					
5/1000	0.439278	0.437146	0.000000	93.9	0.2844
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 5 Val Loss: 0.437146)					
6/1000	0.432982	0.437445	0.000000	86.7	0.2866
7/1000	0.428222	0.436713	0.000000	83.1	0.2863
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 7 Val Loss: 0.436713)					
8/1000	0.424491	0.435696	0.000000	81.1	0.2855
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 8 Val Loss: 0.435696)					
9/1000	0.421421	0.434014	0.000000	80.3	0.2832
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 9 Val Loss: 0.434014)					
10/1000	0.418880	0.432588	0.000000	79.6	0.2814
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 10 Val Loss: 0.432588)					
11/1000	0.416709	0.431104	0.000000	79.3	0.2796
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 11 Val Loss: 0.431104)					
12/1000	0.414752	0.429536	0.000000	79.1	0.2774
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 12 Val Loss: 0.429536)					
13/1000	0.413038	0.428370	0.000000	78.9	0.2758
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					
Mejor modelo guardado (época 13 Val Loss: 0.428370)					
14/1000	0.411505	0.427240	0.000000	78.6	0.2744
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma					

Mejor modelo guardado (época 14 | Val Loss: 0.427240)
 Época 15/1000 | Train: 0.410121 | Val: 0.426191 | L_aux: 0.000000 | L0: 78.5 | w2048=0.2731
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 15 | Val Loss: 0.426191)
 Época 16/1000 | Train: 0.408848 | Val: 0.425135 | L_aux: 0.000000 | L0: 78.3 | w2048=0.2716
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 16 | Val Loss: 0.425135)
 Época 17/1000 | Train: 0.407707 | Val: 0.424747 | L_aux: 0.000000 | L0: 77.7 | w2048=0.2713
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 17 | Val Loss: 0.424747)
 Época 18/1000 | Train: 0.406681 | Val: 0.423822 | L_aux: 0.000000 | L0: 77.8 | w2048=0.2701
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 18 | Val Loss: 0.423822)
 Época 19/1000 | Train: 0.405754 | Val: 0.423368 | L_aux: 0.000000 | L0: 77.5 | w2048=0.2697
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 19 | Val Loss: 0.423368)
 Época 20/1000 | Train: 0.404931 | Val: 0.423014 | L_aux: 0.000000 | L0: 77.1 | w2048=0.2694
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 20 | Val Loss: 0.423014)
 Época 21/1000 | Train: 0.404152 | Val: 0.422242 | L_aux: 0.000000 | L0: 77.4 | w2048=0.2686
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 21 | Val Loss: 0.422242)
 Época 22/1000 | Train: 0.403428 | Val: 0.421366 | L_aux: 0.000000 | L0: 77.8 | w2048=0.2676
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 22 | Val Loss: 0.421366)
 Época 23/1000 | Train: 0.402781 | Val: 0.420875 | L_aux: 0.000000 | L0: 77.8 | w2048=0.2671
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 23 | Val Loss: 0.420875)
 Época 24/1000 | Train: 0.402157 | Val: 0.420162 | L_aux: 0.000000 | L0: 78.2 | w2048=0.2663
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 24 | Val Loss: 0.420162)
 Época 25/1000 | Train: 0.401589 | Val: 0.420006 | L_aux: 0.000000 | L0: 78.0 | w2048=0.2664
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 25 | Val Loss: 0.420006)
 Época 26/1000 | Train: 0.401043 | Val: 0.419514 | L_aux: 0.000000 | L0: 78.1 | w2048=0.2658
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 26 | Val Loss: 0.419514)
 Época 27/1000 | Train: 0.400523 | Val: 0.418806 | L_aux: 0.000000 | L0: 78.6 | w2048=0.2649
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 27 | Val Loss: 0.418806)
 Época 28/1000 | Train: 0.400068 | Val: 0.418809 | L_aux: 0.000000 | L0: 78.3 | w2048=0.2652
 Época 29/1000 | Train: 0.399613 | Val: 0.418366 | L_aux: 0.000000 | L0: 78.4 | w2048=0.2647
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 29 | Val Loss: 0.418366)
 Época 30/1000 | Train: 0.399172 | Val: 0.418102 | L_aux: 0.000000 | L0: 78.5 | w2048=0.2645
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 30 | Val Loss: 0.418102)
 Época 31/1000 | Train: 0.398772 | Val: 0.417517 | L_aux: 0.000000 | L0: 78.8 | w2048=0.2637
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 31 | Val Loss: 0.417517)
 Época 32/1000 | Train: 0.398419 | Val: 0.417117 | L_aux: 0.000000 | L0: 79.0 | w2048=0.2632
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 32 | Val Loss: 0.417117)
 Época 33/1000 | Train: 0.398031 | Val: 0.417003 | L_aux: 0.000000 | L0: 78.9 | w2048=0.2632
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 33 | Val Loss: 0.417003)
 Época 34/1000 | Train: 0.397686 | Val: 0.416495 | L_aux: 0.000000 | L0: 79.3 | w2048=0.2625
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 34 | Val Loss: 0.416495)
 Época 35/1000 | Train: 0.397347 | Val: 0.416388 | L_aux: 0.000000 | L0: 79.2 | w2048=0.2624
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 35 | Val Loss: 0.416388)
 Época 36/1000 | Train: 0.397024 | Val: 0.416059 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2622
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 36 | Val Loss: 0.416059)
 Época 37/1000 | Train: 0.396717 | Val: 0.415493 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2613
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 37 | Val Loss: 0.415493)
 Época 38/1000 | Train: 0.396426 | Val: 0.415949 | L_aux: 0.000000 | L0: 79.2 | w2048=0.2621
 Época 39/1000 | Train: 0.396158 | Val: 0.415254 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2611
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 39 | Val Loss: 0.415254)
 Época 40/1000 | Train: 0.395881 | Val: 0.415190 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2610
 Época 41/1000 | Train: 0.395633 | Val: 0.415019 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2609
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 41 | Val Loss: 0.415019)
 Época 42/1000 | Train: 0.395371 | Val: 0.414723 | L_aux: 0.000000 | L0: 80.0 | w2048=0.2604
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 42 | Val Loss: 0.414723)
 Época 43/1000 | Train: 0.395124 | Val: 0.414984 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2609
 Época 44/1000 | Train: 0.394898 | Val: 0.414736 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2606
 Época 45/1000 | Train: 0.394664 | Val: 0.414513 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2603
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 45 | Val Loss: 0.414513)
 Época 46/1000 | Train: 0.394476 | Val: 0.414329 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2600
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 46 | Val Loss: 0.414329)
 Época 47/1000 | Train: 0.394261 | Val: 0.414201 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2598
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 47 | Val Loss: 0.414201)
 Época 48/1000 | Train: 0.394068 | Val: 0.414116 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2598
 Época 49/1000 | Train: 0.393875 | Val: 0.413518 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2588
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 49 | Val Loss: 0.413518)
 Época 50/1000 | Train: 0.393688 | Val: 0.414112 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2598
 Época 51/1000 | Train: 0.393521 | Val: 0.413936 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2596
 Época 52/1000 | Train: 0.393339 | Val: 0.413733 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2593
 Época 53/1000 | Train: 0.393180 | Val: 0.413674 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2592
 Época 54/1000 | Train: 0.393042 | Val: 0.413614 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2591
 Época 55/1000 | Train: 0.392854 | Val: 0.413870 | L_aux: 0.000000 | L0: 79.3 | w2048=0.2594
 Época 56/1000 | Train: 0.392705 | Val: 0.413437 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2588
 Época 57/1000 | Train: 0.392557 | Val: 0.412807 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2578
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 57 | Val Loss: 0.412807)
 Época 58/1000 | Train: 0.392400 | Val: 0.413507 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2590
 Época 59/1000 | Train: 0.392268 | Val: 0.413670 | L_aux: 0.000000 | L0: 79.2 | w2048=0.2592
 Época 60/1000 | Train: 0.392138 | Val: 0.412849 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2579
 Época 61/1000 | Train: 0.391997 | Val: 0.413016 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2581
 Época 62/1000 | Train: 0.391878 | Val: 0.413028 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2581
 Época 63/1000 | Train: 0.391759 | Val: 0.412525 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2573
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 63 | Val Loss: 0.412525)
 Época 64/1000 | Train: 0.391631 | Val: 0.412848 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2579
 Época 65/1000 | Train: 0.391511 | Val: 0.412527 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2574
 Época 66/1000 | Train: 0.391393 | Val: 0.412755 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2577
 Época 67/1000 | Train: 0.391318 | Val: 0.412382 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2571
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 67 | Val Loss: 0.412382)
 Época 68/1000 | Train: 0.391170 | Val: 0.412615 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2575
 Época 69/1000 | Train: 0.391038 | Val: 0.412768 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2577
 Época 70/1000 | Train: 0.390949 | Val: 0.412585 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2574
 Época 71/1000 | Train: 0.390820 | Val: 0.412674 | L_aux: 0.000000 | L0: 79.4 | w2048=0.2575
 Época 72/1000 | Train: 0.390718 | Val: 0.412313 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2569
 Época 73/1000 | Train: 0.390622 | Val: 0.412066 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2566
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 73 | Val Loss: 0.412066)
 Época 74/1000 | Train: 0.390545 | Val: 0.412301 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2569
 Época 75/1000 | Train: 0.390436 | Val: 0.412219 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2568
 Época 76/1000 | Train: 0.390345 | Val: 0.411782 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2561

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 76 | Val Loss: 0.411782)

Época 77/1000 | Train: 0.390257 | Val: 0.412167 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2567

Época 78/1000 | Train: 0.390168 | Val: 0.411409 | L_aux: 0.000000 | L0: 80.6 | w2048=0.2555

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 78 | Val Loss: 0.411409)

Época 79/1000 | Train: 0.390088 | Val: 0.412204 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2567

Época 80/1000 | Train: 0.389994 | Val: 0.411498 | L_aux: 0.000000 | L0: 80.3 | w2048=0.2556

Época 81/1000 | Train: 0.389904 | Val: 0.411693 | L_aux: 0.000000 | L0: 80.0 | w2048=0.2559

Época 82/1000 | Train: 0.389823 | Val: 0.411788 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2560

Época 83/1000 | Train: 0.389751 | Val: 0.411281 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2552

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 83 | Val Loss: 0.411281)

Época 84/1000 | Train: 0.389672 | Val: 0.411477 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2555

Época 85/1000 | Train: 0.389584 | Val: 0.411583 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2556

Época 86/1000 | Train: 0.389511 | Val: 0.411972 | L_aux: 0.000000 | L0: 79.4 | w2048=0.2562

Época 87/1000 | Train: 0.389418 | Val: 0.411575 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2556

Época 88/1000 | Train: 0.389341 | Val: 0.411345 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2552

Época 89/1000 | Train: 0.389290 | Val: 0.411451 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2554

Época 90/1000 | Train: 0.389237 | Val: 0.411567 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2556

Época 91/1000 | Train: 0.389167 | Val: 0.411243 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2550

Época 92/1000 | Train: 0.389095 | Val: 0.411666 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2556

Época 93/1000 | Train: 0.389024 | Val: 0.411166 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2548

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 93 | Val Loss: 0.411166)

Época 94/1000 | Train: 0.388952 | Val: 0.411131 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2548

Época 95/1000 | Train: 0.388871 | Val: 0.411457 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2553

Época 96/1000 | Train: 0.388829 | Val: 0.411030 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2546

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 96 | Val Loss: 0.411030)

Época 97/1000 | Train: 0.388756 | Val: 0.411230 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2549

Época 98/1000 | Train: 0.388705 | Val: 0.411192 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2548

Época 99/1000 | Train: 0.388638 | Val: 0.410602 | L_aux: 0.000000 | L0: 80.6 | w2048=0.2538

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 99 | Val Loss: 0.410602)

Época 100/1000 | Train: 0.388591 | Val: 0.410877 | L_aux: 0.000000 | L0: 80.3 | w2048=0.2543

Época 101/1000 | Train: 0.388525 | Val: 0.410875 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2543

Época 102/1000 | Train: 0.388461 | Val: 0.411154 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2546

Época 103/1000 | Train: 0.388394 | Val: 0.411237 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2547

Época 104/1000 | Train: 0.388349 | Val: 0.411023 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2544

Época 105/1000 | Train: 0.388302 | Val: 0.411084 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2545

Época 106/1000 | Train: 0.388240 | Val: 0.411495 | L_aux: 0.000000 | L0: 79.2 | w2048=0.2551

Época 107/1000 | Train: 0.388202 | Val: 0.411268 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2547

Época 108/1000 | Train: 0.388137 | Val: 0.410834 | L_aux: 0.000000 | L0: 80.0 | w2048=0.2540
 Época 109/1000 | Train: 0.388080 | Val: 0.410593 | L_aux: 0.000000 | L0: 80.3 | w2048=0.2536
 Época 110/1000 | Train: 0.388046 | Val: 0.410385 | L_aux: 0.000000 | L0: 80.6 | w2048=0.2533
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 110 | Val Loss: 0.410385)
 Época 111/1000 | Train: 0.388008 | Val: 0.410962 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2542
 Época 112/1000 | Train: 0.387961 | Val: 0.410851 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2540
 Época 113/1000 | Train: 0.387912 | Val: 0.410074 | L_aux: 0.000000 | L0: 81.0 | w2048=0.2527
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 113 | Val Loss: 0.410074)
 Época 114/1000 | Train: 0.387865 | Val: 0.410882 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2540
 Época 115/1000 | Train: 0.387826 | Val: 0.410588 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2535
 Época 116/1000 | Train: 0.387769 | Val: 0.410668 | L_aux: 0.000000 | L0: 80.0 | w2048=0.2536
 Época 117/1000 | Train: 0.387735 | Val: 0.410040 | L_aux: 0.000000 | L0: 80.8 | w2048=0.2526
 Época 118/1000 | Train: 0.387683 | Val: 0.410837 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2538
 Época 119/1000 | Train: 0.387635 | Val: 0.410704 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2536
 Época 120/1000 | Train: 0.387613 | Val: 0.410202 | L_aux: 0.000000 | L0: 80.5 | w2048=0.2528
 Época 121/1000 | Train: 0.387566 | Val: 0.410781 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2537
 Época 122/1000 | Train: 0.387508 | Val: 0.410611 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2534
 Época 123/1000 | Train: 0.387483 | Val: 0.410329 | L_aux: 0.000000 | L0: 80.3 | w2048=0.2530
 Época 124/1000 | Train: 0.387426 | Val: 0.410996 | L_aux: 0.000000 | L0: 79.4 | w2048=0.2539
 Época 125/1000 | Train: 0.387411 | Val: 0.410677 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2535
 Época 126/1000 | Train: 0.387362 | Val: 0.410508 | L_aux: 0.000000 | L0: 80.0 | w2048=0.2532
 Época 127/1000 | Train: 0.387321 | Val: 0.410168 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2526
 Época 128/1000 | Train: 0.387284 | Val: 0.410195 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2526
 Época 129/1000 | Train: 0.387234 | Val: 0.410285 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2528
 Época 130/1000 | Train: 0.387209 | Val: 0.410646 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2533
 Época 131/1000 | Train: 0.387165 | Val: 0.411048 | L_aux: 0.000000 | L0: 79.1 | w2048=0.2539
 Época 132/1000 | Train: 0.387130 | Val: 0.410304 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2527
 Época 133/1000 | Train: 0.387097 | Val: 0.409965 | L_aux: 0.000000 | L0: 80.6 | w2048=0.2522
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 133 | Val Loss: 0.409965)
 Época 134/1000 | Train: 0.387058 | Val: 0.410380 | L_aux: 0.000000 | L0: 80.0 | w2048=0.2528
 Época 135/1000 | Train: 0.387014 | Val: 0.410469 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2530
 Época 136/1000 | Train: 0.386996 | Val: 0.409997 | L_aux: 0.000000 | L0: 80.5 | w2048=0.2522
 Época 137/1000 | Train: 0.386955 | Val: 0.410096 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2523
 Época 138/1000 | Train: 0.386923 | Val: 0.410415 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2528
 Época 139/1000 | Train: 0.386879 | Val: 0.410425 | L_aux: 0.000000 | L0: 79.8 | w2048=0.2527
 Época 140/1000 | Train: 0.386850 | Val: 0.410566 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2530
 Época 141/1000 | Train: 0.386809 | Val: 0.410506 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2528
 Época 142/1000 | Train: 0.386783 | Val: 0.410658 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2531
 Época 143/1000 | Train: 0.386749 | Val: 0.410460 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2528
 Época 144/1000 | Train: 0.386709 | Val: 0.410156 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2523

Época 145/1000 | Train: 0.386672 | Val: 0.410147 | L_aux: 0.000000 | L0: 80.0 | w2048=0.2522
 Época 146/1000 | Train: 0.386652 | Val: 0.409807 | L_aux: 0.000000 | L0: 80.6 | w2048=0.2517
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 146 | Val Loss: 0.409807)
 Época 147/1000 | Train: 0.386620 | Val: 0.409655 | L_aux: 0.000000 | L0: 80.8 | w2048=0.2514
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 147 | Val Loss: 0.409655)
 Época 148/1000 | Train: 0.386588 | Val: 0.410315 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2524
 Época 149/1000 | Train: 0.386569 | Val: 0.409978 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2519
 Época 150/1000 | Train: 0.386537 | Val: 0.409665 | L_aux: 0.000000 | L0: 80.7 | w2048=0.2513
 Época 151/1000 | Train: 0.386506 | Val: 0.410017 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2519
 Época 152/1000 | Train: 0.386466 | Val: 0.410793 | L_aux: 0.000000 | L0: 79.1 | w2048=0.2531
 Época 153/1000 | Train: 0.386440 | Val: 0.411344 | L_aux: 0.000000 | L0: 78.3 | w2048=0.2539
 Época 154/1000 | Train: 0.386405 | Val: 0.409976 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2518
 Época 155/1000 | Train: 0.386364 | Val: 0.410329 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2523
 Época 156/1000 | Train: 0.386357 | Val: 0.409817 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2515
 Época 157/1000 | Train: 0.386314 | Val: 0.410426 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2524
 Época 158/1000 | Train: 0.386286 | Val: 0.409841 | L_aux: 0.000000 | L0: 80.3 | w2048=0.2515
 Época 159/1000 | Train: 0.386259 | Val: 0.410066 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2518
 Época 160/1000 | Train: 0.386237 | Val: 0.410411 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2523
 Época 161/1000 | Train: 0.386232 | Val: 0.409913 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2515
 Época 162/1000 | Train: 0.386176 | Val: 0.409622 | L_aux: 0.000000 | L0: 80.5 | w2048=0.2511
 Época 163/1000 | Train: 0.386163 | Val: 0.410230 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2520
 Época 164/1000 | Train: 0.386136 | Val: 0.409595 | L_aux: 0.000000 | L0: 80.5 | w2048=0.2510
 Época 165/1000 | Train: 0.386096 | Val: 0.410587 | L_aux: 0.000000 | L0: 79.1 | w2048=0.2525
 Época 166/1000 | Train: 0.386072 | Val: 0.409890 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2514
 Época 167/1000 | Train: 0.386052 | Val: 0.409588 | L_aux: 0.000000 | L0: 80.5 | w2048=0.2509

Early stopping en época 167 (mejor: época 147)

Mejor época: 147 | Mejor Val Loss: 0.409655

8. Guardar Modelo

```
# Guardar checkpoint final
checkpoint = {
    'model_state_dict':    model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'config': {
        'architecture':    cfg['architecture'],
        'd_in':             cfg['d_in'],
```

```

        'd_sae':          cfg['d_sae'],
        'k':              cfg['k'],
        'matryoshka_widths': cfg['matryoshka_widths'],
        'lr':             cfg['lr'],
        'hook_name':      cfg['hook_name'],
    },
    'history': history,
}

final_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final',
    extras_id=extras_id
)
checkpoint_path = save_dir / final_model_name
torch.save(checkpoint, checkpoint_path)
print(f'Modelo guardado: {checkpoint_path}')

save_checkpoint(
    model,
    base_path=NAMING_META['base_path'],
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final',
    extras=NAMING_META['extras'],
    config=cfg,
    history=history,
    optimizer=optimizer
)

```

Modelo guardado: saved_model\sae_matryoshkabatchtopk_matryoshkabatchtopk_l6_5000g_ex2_final.json

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_matryoshkabatchtopk_l6_5000g_ex2_final.json

WindowsPath('C:/Users/Esposa/Documents/Repos/sae-othello-gpt/layer_06/models/sae_matryoshkabatchtopk_l6_5000g_ex2_final.json')

9. Métricas básicas del modelo

```
metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

training_metrics = {
    # Calidad de reconstrucción (loss total = media de MSE en todos los anchos)
    'val_total_loss_final': history['val_total_loss'][-1],
    'val_total_loss_best': best_val_loss,
    # MSE por ancho matryoshka (último epoch)
    **{f'val_mse_w{w}_final': history[f'val_mse_w{w}'][-1] for w in WIDTHS},
    # Sparsity (L0 del ancho máximo)
    'val_l0_final': history['val_l0_sparsity'][-1],
    'val_fraction_active_final': history['val_fraction_active'][-1],
    # Contexto del entrenamiento
    'best_epoch': best_epoch,
    'total_epochs': len(history['train_total_loss']),
    'train_total_loss_final': history['train_total_loss'][-1],
    'train_l0_final': history['train_l0_sparsity'][-1],
    # Hiperparámetros MatryoshkaBatchTopK
    'architecture': cfg['architecture'],
    'k': cfg['k'],
    'matryoshka_widths': cfg['matryoshka_widths'],
    'd_sae': cfg['d_sae'],
    'd_in': cfg['d_in'],
    'expansion_factor': cfg['d_sae'] // cfg['d_in'],
    'hook_name': cfg['hook_name'],
}

os.makedirs(metrics_dir, exist_ok=True)
metrics_path = Path(metrics_dir) / 'training_metrics.json'
with open(metrics_path, 'w') as f:
    json.dump(training_metrics, f, indent=2)

print(f'Métricas guardadas: {metrics_path}')
print(pd.DataFrame(
```

```
list(training_metrics.items()), columns=['Métrica', 'Valor']
).to_string(index=False))
```

Métricas guardadas: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\s

Métrica	Valor
val_total_loss_final	0.409588
val_total_loss_best	0.409655
val_mse_w2048_final	0.25093
val_mse_w4096_final	0.224211
val_mse_w8192_final	0.209287
val_l0_final	80.540218
val_fraction_active_final	0.009832
best_epoch	147
total_epochs	167
train_total_loss_final	0.386052
train_l0_final	81.0
architecture	matryoshkabatchtopk
k	81
matryoshka_widths	[2048, 4096, 8192]
d_sae	8192
d_in	512
expansion_factor	16
hook_name	blocks.5.hook_resid_post

10. Visualización de Resultados

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Total Nested Loss
axes[0].plot(history['train_total_loss'], label='Train', color='steelblue', alpha=0.8)
axes[0].plot(history['val_total_loss'], label='Validación', color='tomato', alpha=0.8)
if best_epoch > 0:
    axes[0].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7,
                    label=f'Best (época {best_epoch})')
axes[0].set_title(f'Total Nested Loss ( $\Sigma$  MSE en {len(WIDTHS)} anchos)')
axes[0].set_xlabel('Época')
axes[0].set_ylabel('Loss total')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
```

```

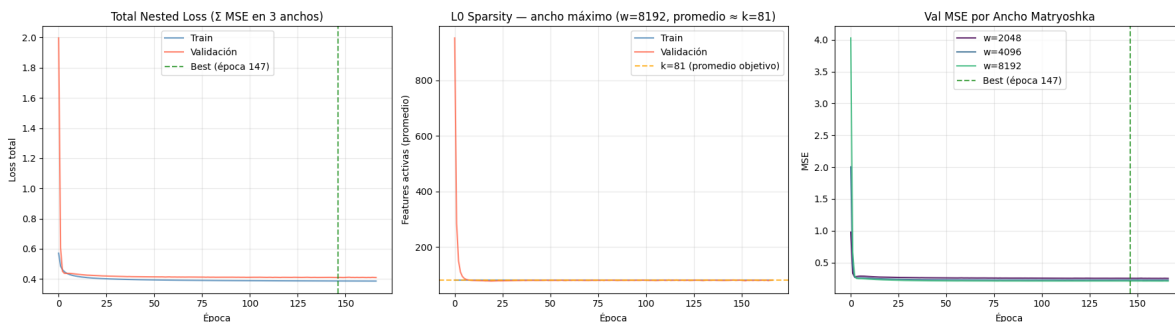
# L0 sparsity (ancho máximo, L0 promedio k)
axes[1].plot(history['train_l0_sparsity'], label='Train', color='steelblue', alpha=0.8)
axes[1].plot(history['val_l0_sparsity'], label='Validación', color='tomato', alpha=0.8)
axes[1].axhline(cfg["k"], color='orange', linestyle='--', alpha=0.8,
                label=f'k={cfg["k"]} (promedio objetivo)')
axes[1].set_title(f'L0 Sparsity - ancho máximo (w={cfg["d_sae"]}, promedio k={cfg["k"]})')
axes[1].set_xlabel('Época')
axes[1].set_ylabel('Features activas (promedio)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# MSE por ancho matryoshka (validación)
colors_w = plt.cm.viridis([i / len(WIDTHS) for i in range(len(WIDTHS))])
for w, color in zip(WIDTHS, colors_w):
    axes[2].plot(history[f'val_mse_w{w}'], label=f'w={w}', color=color, alpha=0.85)
if best_epoch > 0:
    axes[2].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7,
                  label=f'Best (época {best_epoch})')
axes[2].set_title('Val MSE por Ancho Matryoshka')
axes[2].set_xlabel('Época')
axes[2].set_ylabel('MSE')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(plots_dir / 'training_curves.png', dpi=300, bbox_inches='tight')
plt.show()

print('\n' + '='*50)
print('Gráficas guardadas exitosamente')
print('='*50)

```



```
=====
Gráficas guardadas exitosamente
=====
```

12. Generar PDF con Quarto

Una vez ejecutado todo el notebook, ejecutar el siguiente comando en la terminal para generar el PDF con Quarto:

```
exp_dir = get_experiment_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'training',
    extras_id=extras_id
)

notebook_name = 'train_sae_matryoska-batch-topk.ipynb'
output_path = exp_dir / pdf_name

print(f'Generando PDF con Quarto...')
print(f'Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{exp_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)
```

```
if result.returncode == 0:
    print(f'PDF generado exitosamente: {output_path}')
else:
    print(f'Error al generar PDF:')
    print(result.stderr)
```

Generando PDF con Quarto...

Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae

PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae